Laboratorio #11.2:

Objetivos:

- Desplegar DefectDojo localmente con Docker Compose
- Configurar la jerarquía Organization (Product Type) → Asset (Product) → Engagement dentro de la plataforma
- Automatizar la ingesta de hallazgos de Gitleaks, TruffleHog, Trivy y Semgrep desde un job de GitLab CI/CD usando la API REST (reimport-scan)
- Hacer que cada job de escaneo falle en rojo ante hallazgos sin frenar la carga a DefectDojo
- Interpretar el dashboard de deuda técnica de seguridad y gestionar hallazgos (Risk Acceptance, False Positive)

Para resolver este laboratorio es necesario haber resuelto el laboratorio 11.1 (pipeline de `juice-shop-devsecops` con Gitleaks, TruffleHog, Trivy y Semgrep):

Guía de laboratorio 11.1:

- https://github.com/utec-devops-2026/laboratorios/blob/main/laboratorios_devsecops/guia_secret_scanning_sca_sast.md

Guía de laboratorio 11.2:

- https://github.com/utec-devops-2026/laboratorios/blob/main/laboratorios_devsecops/guia_defect_dojo.md

Requisitos:

- Docker Desktop con al menos **8 GB RAM** asignados (DefectDojo levanta 6 contenedores; el primer arranque tarda 2–5 min)
- Cuenta gratuita en `ngrok.com`: los runners SaaS de GitLab no alcanzan `localhost`, hay que exponer DefectDojo con un túnel público
- Nombres en la UI de DefectDojo 3.x: **Product Type** → **Organization** y **Product** → **Asset**. La API conserva `/api/v2/products/`.

Entregables

1. **URL de repositorio** (GitLab).
   - https://gitlab.com/aaron.timana/juice-shop-devsecops-20262

2. Subir su repositorio a GitLab. Brindar acceso a usuario (@pytuxi). Es el mismo repo del laboratorio 11.1 con el stage `defectdojo` agregado.

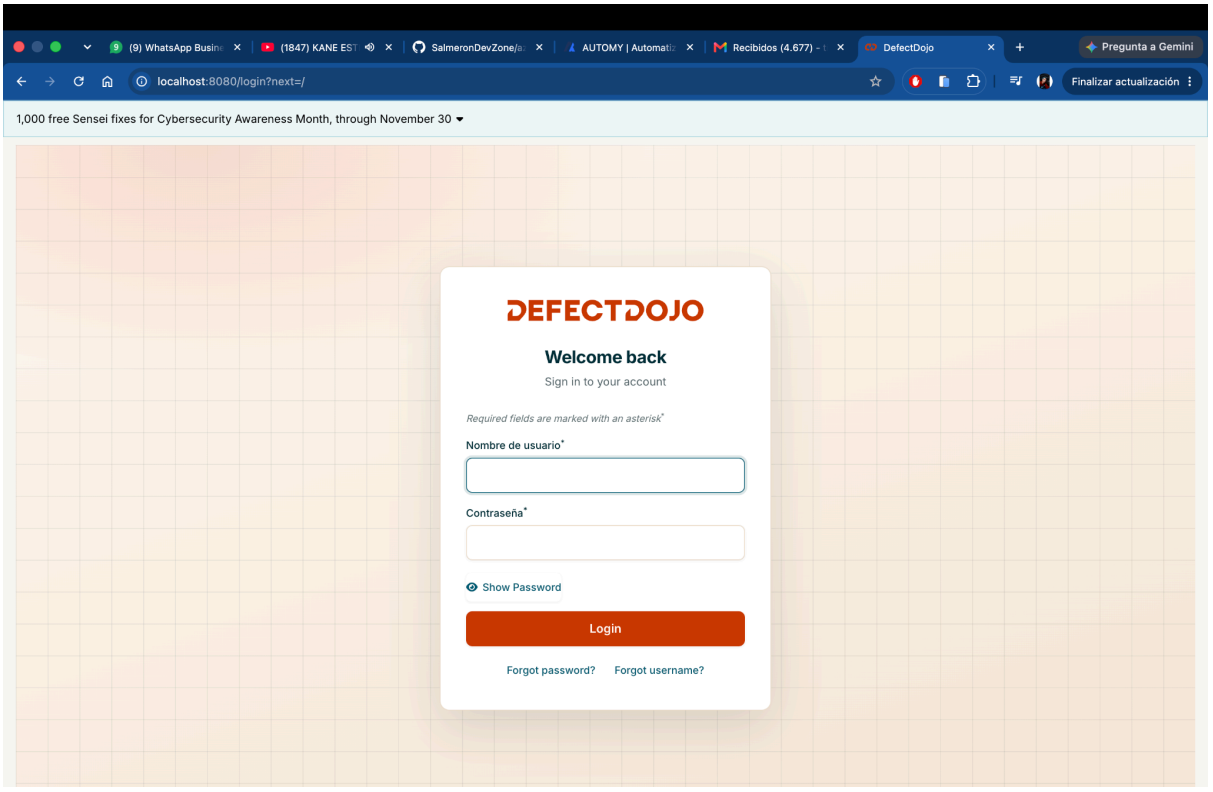
Estructura del repo:

```
juice-shop-devsecops/
├── .gitlab-ci.yml          # + stage defectdojo
```

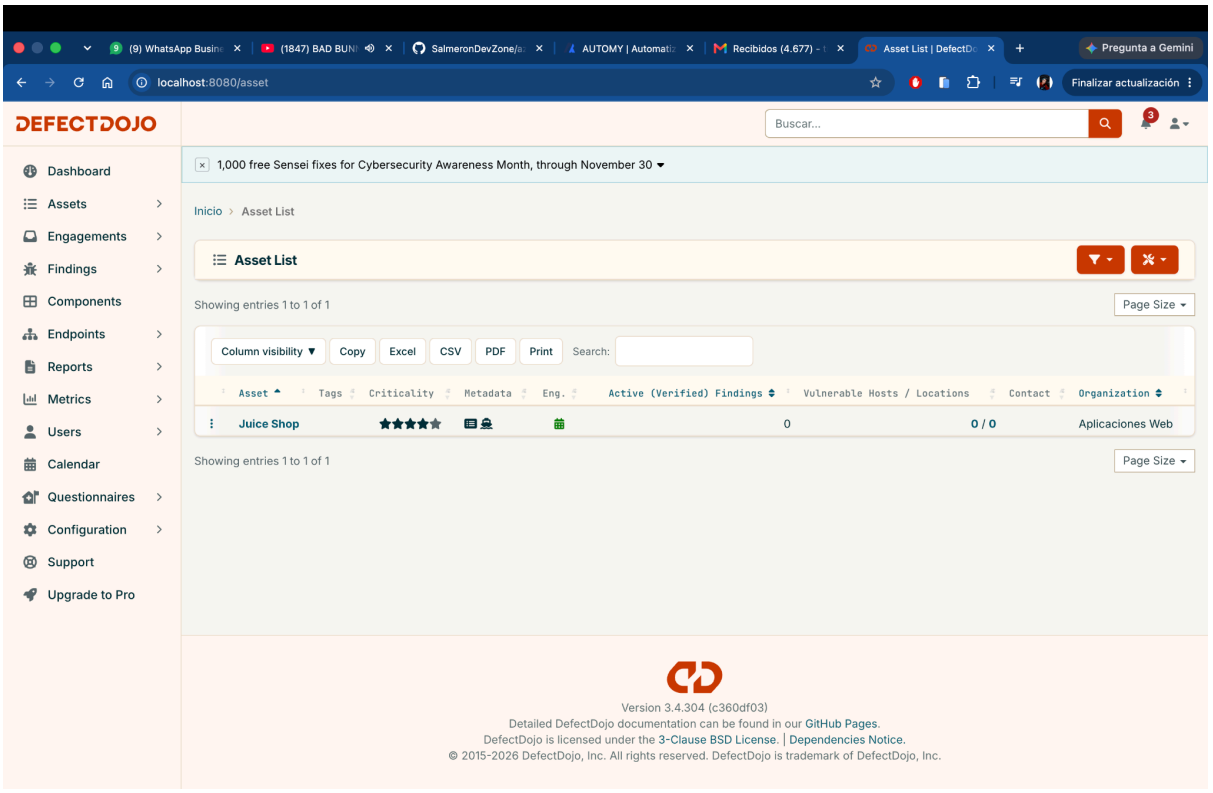
```
│                              # scans: artifacts.when always
├── scripts/
│   └── upload_to_defectdojo.sh  # reimport-scan
├── .pre-commit-config.yaml
├── package.json
├── package-lock.json          # necesario para Trivy SCA
├── routes/ lib/ models/ frontend/
└── reports/                   # local, NO se commitea
```

3. **Parte 1 – Instalación:** captura de `docker compose ps` con todos los servicios en Up e `initializer` en Exited (0), y captura del login en http://localhost:8080.

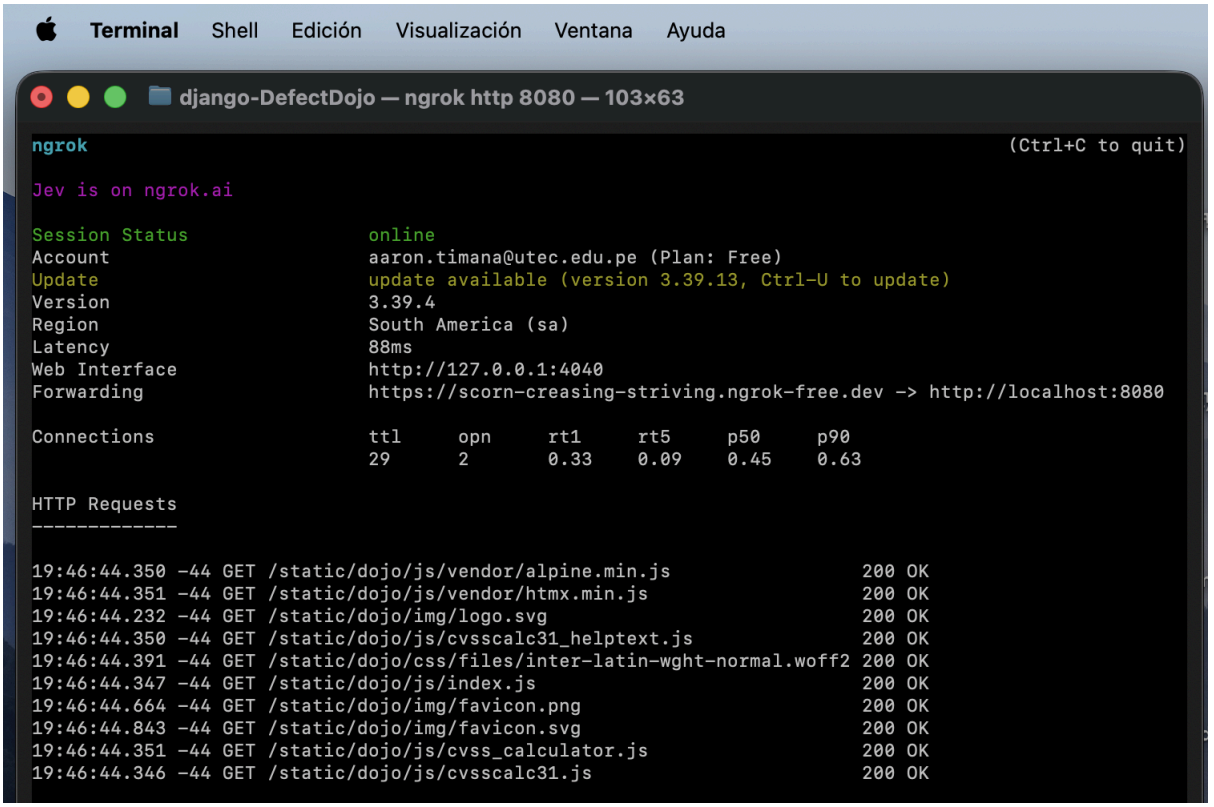
```
usuario@MacBookPro django-DefectDojo % docker compose -f docker-compose.yml -f docker-compose.override.
dev.yml ps -a
NAME                                    IMAGE
                                       COMMAND                  SERVICE            CREATED         STATU
[S                      PORTS                                                                           ]
django-defectdojo-celerybeat-1          registry.defectdojo.com/defectdojo/defectdojo-django:latest
                                       "/wait-for-it.sh pos…"   celerybeat         3 minutes ago   Up 2
minutes
django-defectdojo-celeryworker-1        registry.defectdojo.com/defectdojo/defectdojo-django:latest
                                       "/wait-for-it.sh pos…"   celeryworker       3 minutes ago   Up 2
minutes
django-defectdojo-initializer-1         registry.defectdojo.com/defectdojo/defectdojo-django:latest
                                       "/wait-for-it.sh pos…"   initializer        3 minutes ago   Exite
d (0) 2 minutes ago
django-defectdojo-mailhog-1             mailhog/mailhog:v1.0.1@sha256:8d76a3d4ffa32a3661311944007a415332
c4bb855657f4f6c57996405c009bea          "/bin/sh -c 'MailHog…"   mailhog            3 minutes ago   Up 3
minutes                 0.0.0.0:1025->1025/tcp, [::]:1025->1025/tcp, 0.0.0.0:8025->8025/tcp, [::]:8025->8
025/tcp
django-defectdojo-nginx-1               registry.defectdojo.com/defectdojo/defectdojo-nginx:latest
                                       "/entrypoint-nginx.sh"   nginx              3 minutes ago   Up 2
minutes                 0.0.0.0:8080->8080/tcp, [::]:8080->8080/tcp, 0.0.0.0:8443->8443/tcp, [::]:8443->8
443/tcp
django-defectdojo-postgres-1            postgres:18.6-alpine@sha256:77f585114c32fbca283dc835b0596f4e52b5
1b4c6662d7810b2f4084f60a1873           "docker-entrypoint.s…"   postgres           3 minutes ago   Up 3
minutes                 0.0.0.0:5432->5432/tcp, [::]:5432->5432/tcp
django-defectdojo-uwsgi-1               registry.defectdojo.com/defectdojo/defectdojo-django:latest
                                       "/wait-for-it.sh pos…"   uwsgi              3 minutes ago   Up 2
minutes
django-defectdojo-valkey-1              valkey/valkey:9.1.2-alpine@sha256:48332870af354a799964c0012ae119
4a0bf2bf894eb508f945810596dc2d8d11      "tini -- docker-entr…"   valkey             3 minutes ago   Up 3
minutes                 6379/tcp
django-defectdojo-webhook.endpoint-1    mccutchen/go-httpbin:2.26.0@sha256:873b4fdb27fe7cc66fa12cb936257
4e1b0bf31df7ca8a71b2502988327a0b8a4     "/usr/bin/go-httpbin"    webhook.endpoint   3 minutes ago   Up 3
minutes                 8080/tcp
usuario@MacBookPro django-DefectDojo %
```

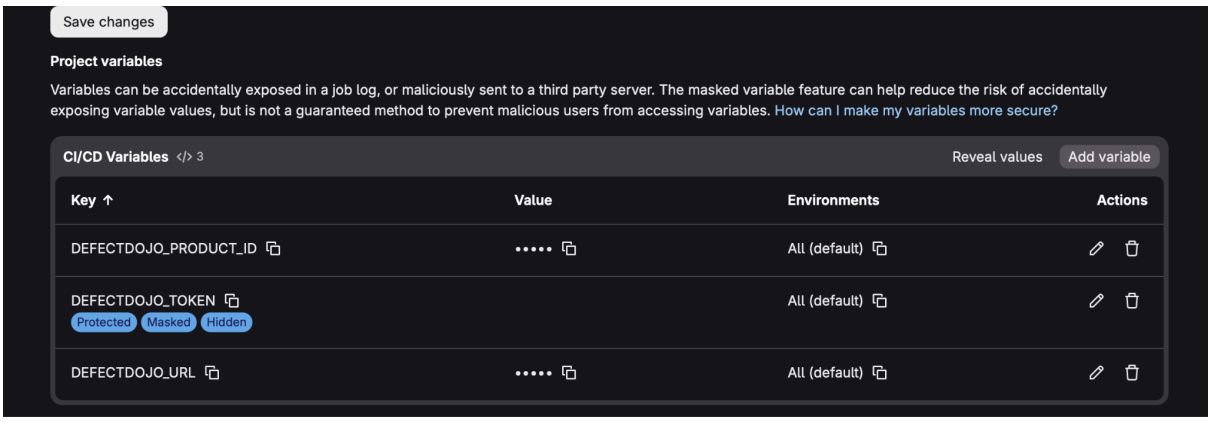


4. **Parte 2 – Configuración:** captura del Asset `Juice Shop` dentro de la Organization `Aplicaciones Web` con su engagement creado (Assets → All Assets → Juice Shop → Engagements).

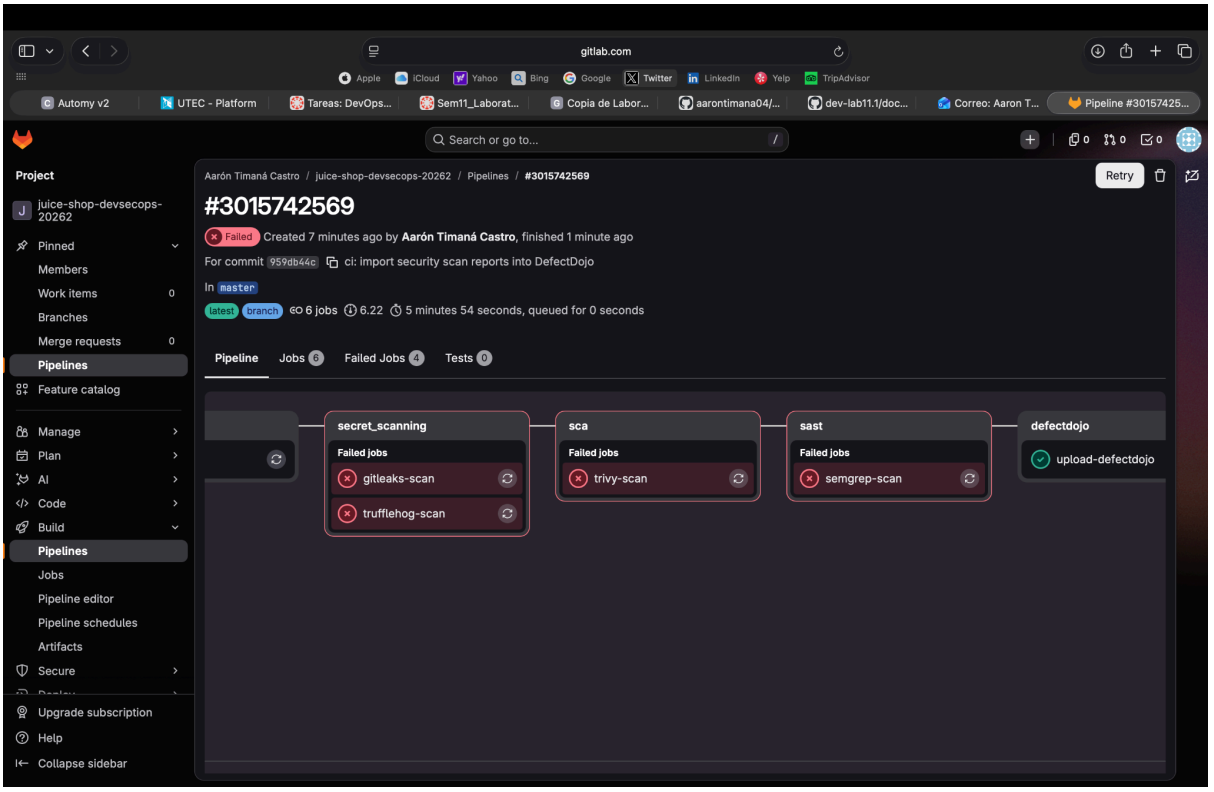


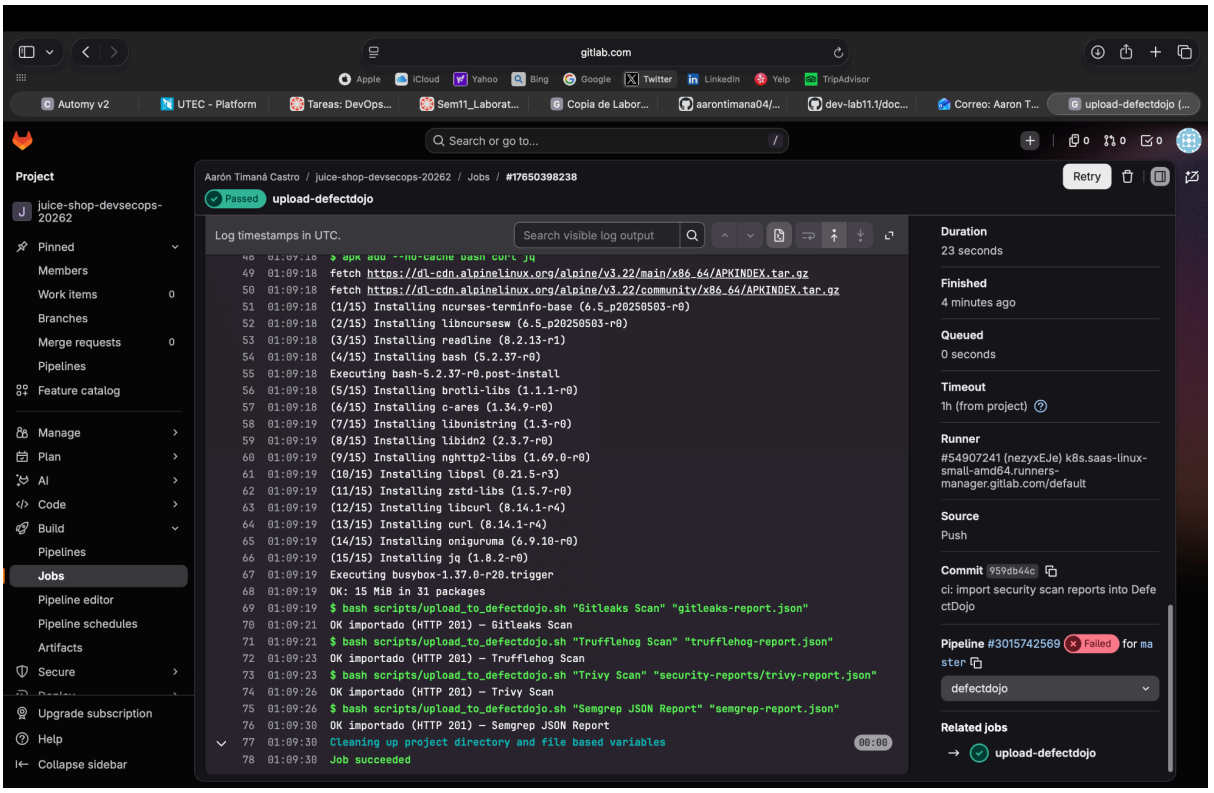
5. **Parte 4 – Automatización GitLab CI/CD:** capturas de
    - Variables CI/CD del proyecto: `DEFECTDOJO_URL` (URL de ngrok), `DEFECTDOJO_TOKEN` (protegida y enmascarada) y `DEFECTDOJO_PRODUCT_ID`; terminal con ngrok corriendo



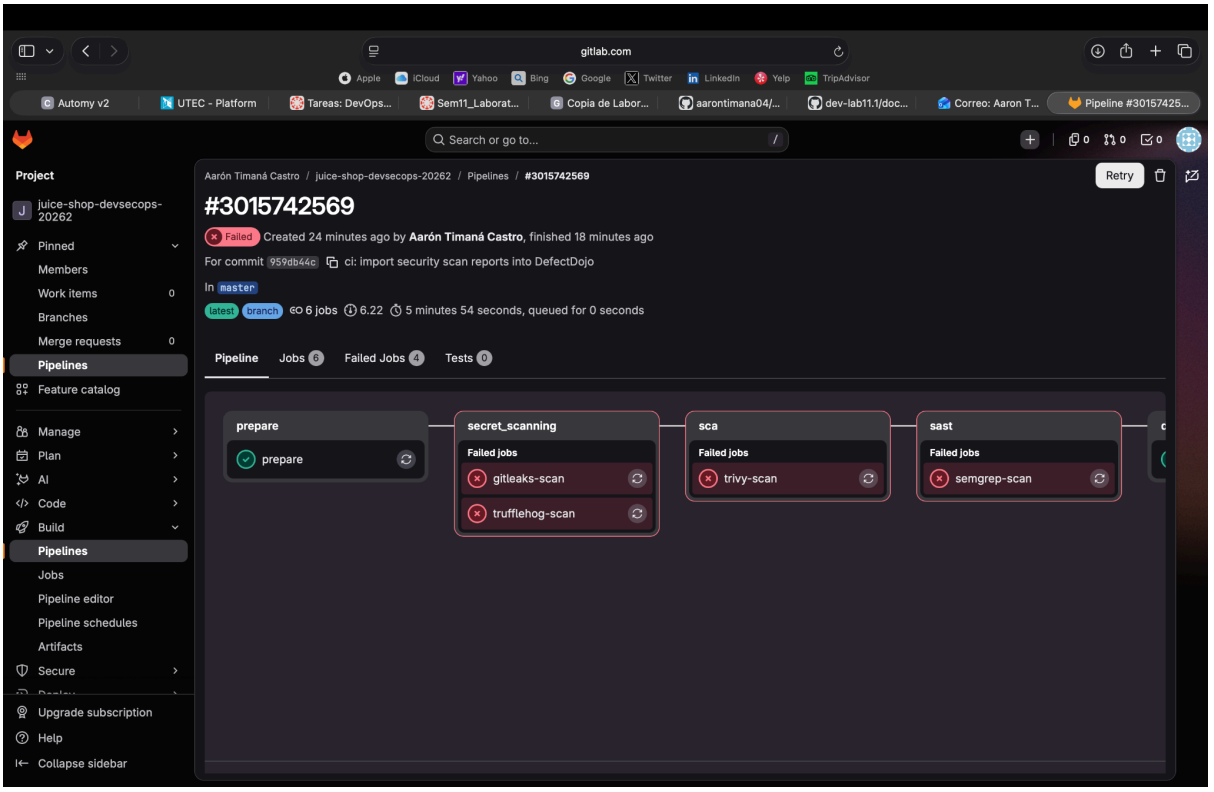


    - Pipeline con el stage `defectdojo` y log del job `upload-defectdojo` mostrando `OK importado (HTTP 201)` para los 4 scan types (Gitleaks Scan, Trufflehog Scan, Trivy Scan, Semgrep JSON Report)

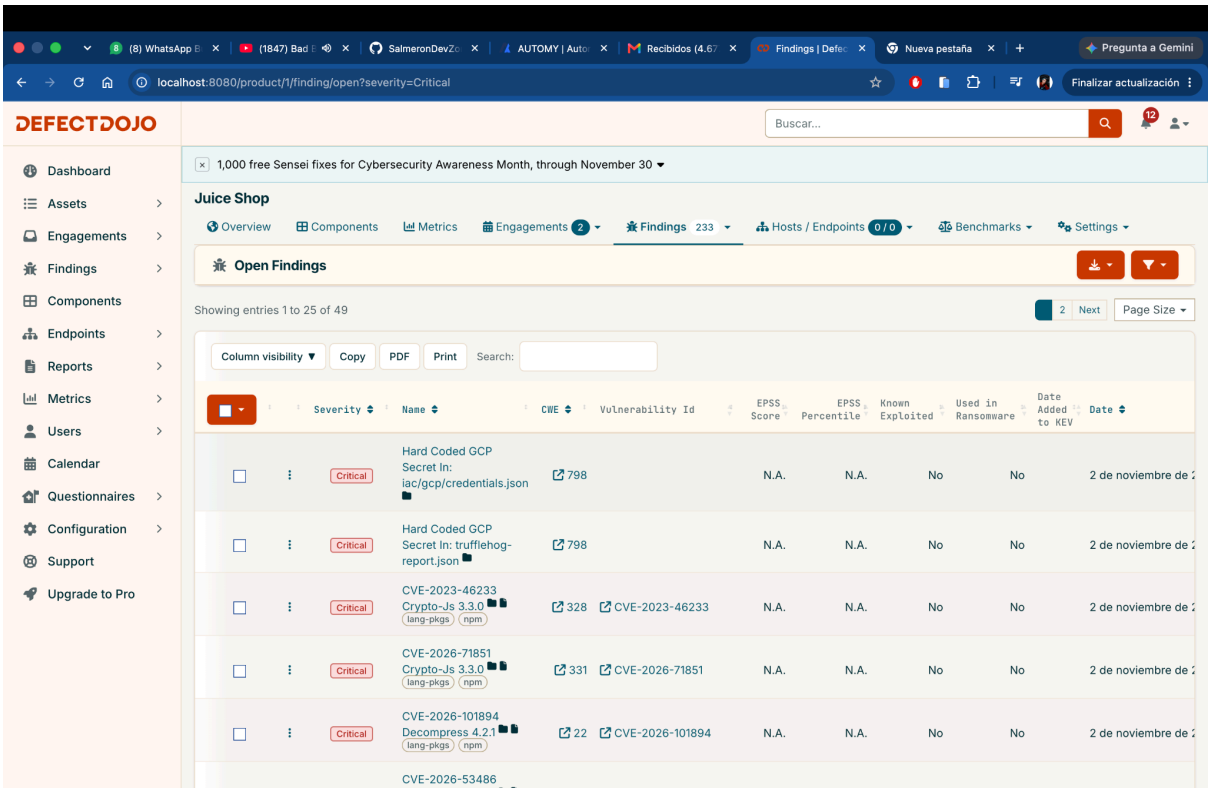


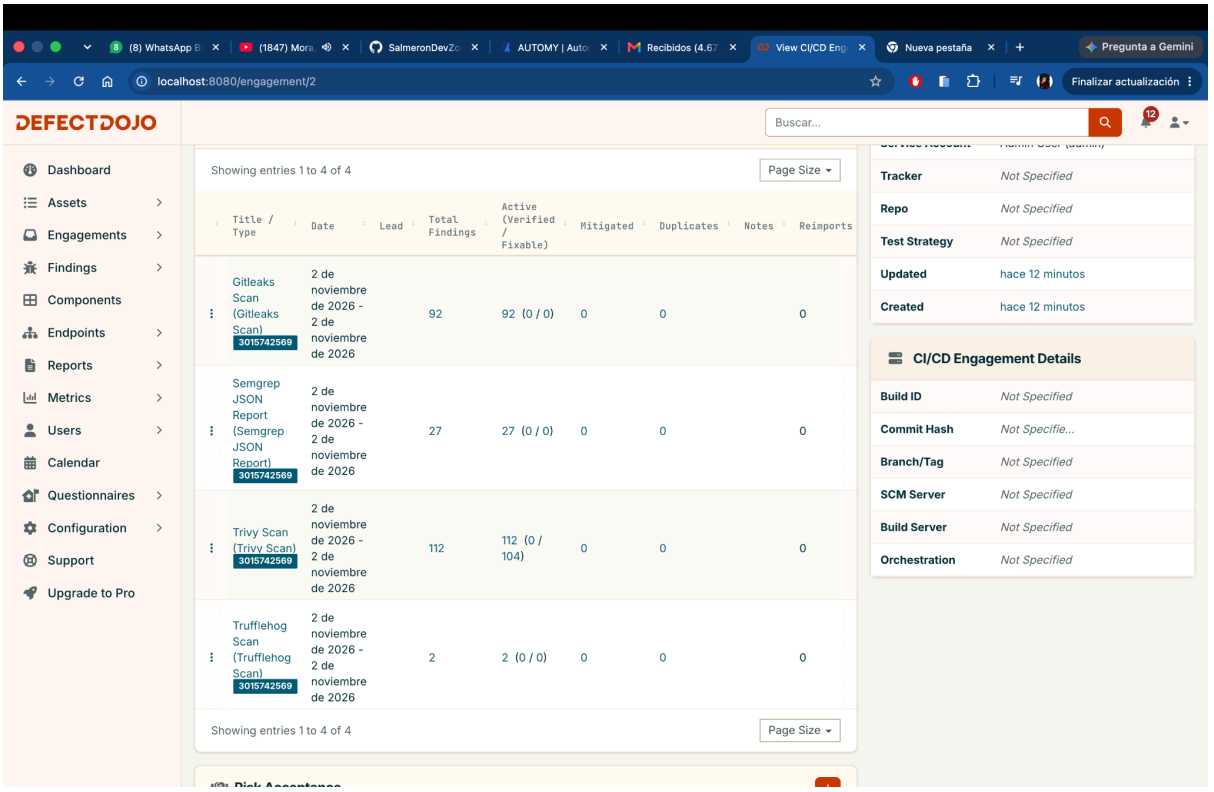


- Parte 4.4: los jobs de escaneo en **rojo (failed)** por hallazgos, con artifacts generados igual, y `upload-defectdojo` ejecutado a pesar de ello



6. **Parte 5 – Dashboard:** capturas de
    - Asset `Juice Shop`: findings por severidad y por herramienta de origen; engagement `CI - <rama>` con sus 4 tests





- Pestaña **Metrics** del asset (open findings por severidad, age of findings)
- Un finding Critical o High gestionado: **Risk Acceptance** con justificación y fecha de expiración, o **False Positive** con nota

7. Preguntas de reflexión:

   - **¿Por qué el runner SaaS de GitLab no puede alcanzar tu DefectDojo en `localhost` y qué riesgos implica exponerlo con ngrok? ¿Cómo lo harías en una empresa?**
     El localhost del runner apunta a su propio entorno, no a mi computadora. Ngrok permite conectarlo con DefectDojo, pero lo expone a Internet. En una empresa usaría una red privada o VPN, HTTPS y acceso restringido.

   - **Con los 4 scanners centralizados, ¿qué hallazgos se repiten entre herramientas (p. ej., Gitleaks vs TruffleHog) y cómo los deduplica DefectDojo?**
     Gitleaks, TruffleHog y Semgrep pueden detectar las mismas credenciales expuestas. DefectDojo compara los hallazgos según sus reglas de deduplicación; reimport-scan permite actualizar resultados sin repetirlos en cada ejecución. Las coincidencias entre herramientas requieren revisar su configuración.